Petterson's Generalised Algorithm

Mutual Exclusion for N Processes

6CM504 — Concurrency and Communication

Exercise 6, Task 2 — Lab Guide

Why do we need Peterson's algorithm?

Dekker's limitation

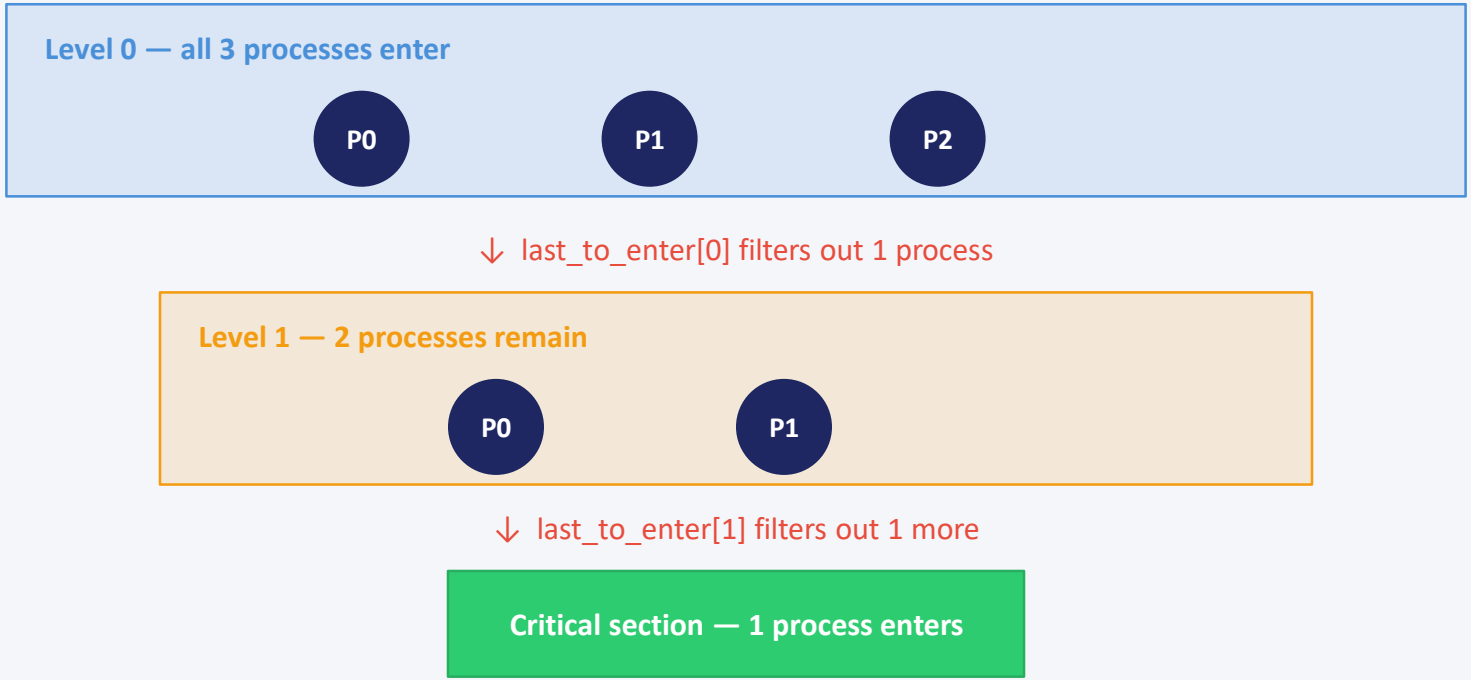
- Only works for 2 processes
- Uses flag[0], flag[1] and turn
- Cannot generalise to N > 2
- Vulnerable to compiler optimisations

Peterson's solution

- Works for arbitrary N processes
- Uses level[N] and last_to_enter[N-1]
- Guarantees mutual exclusion
- Guarantees progress (no blocking)
- Bounded waiting (starvation freedom)

The queue metaphor

Think of it as a series of waiting rooms (levels). With N processes, there are N−1 levels. At each level, not all processes can advance — ultimately only one reaches the critical section.



The variables

`level[N]`

One per process (level[0], level[1], level[2])

Values range from -1 to N-2

level[i] = current "queue position" of process i

level[i] = -1 means "not competing"

`last_to_enter[N-1]`

One per level (last_to_enter[0], last_to_enter[1])

Stores the ID of the last process to enter that level

The "loser" — if you're last, you wait

Ensures at most N-1 pass each gate

Example state (N=3)

`level[0] = 1`	`level[1] = 0`	`level[2] = -1`	`lte[0] = 1`	`lte[1] = 0`

P0 is at level 1, P1 is at level 0, P2 is not competing. P1 was last to enter level 0 (so P1 waits). P0 was last to enter level 1.

The algorithm

```
// Process i wants to enter CS
FOR l = 0 TO N-2
BEGIN
  level[i] := l;
  last_to_enter[l] := i;

  WHILE last_to_enter[l] = i
    AND exists k != i
      where level[k] >= l
  DO nothing  // spin-wait
END

<critical section>

level[i] := -1;  // release
```

Step 1: Announce intent

Write to level[i] and last_to_enter[l] to say "I want in"

Step 2: Spin-wait

Wait until either:
- Someone else enters last, OR
- No one else is at this level or higher

Step 3: Release

Set level[i] = -1 so others know you're done and can proceed

Understanding the spin-wait

The WHILE loop has two conditions — both must be true for process i to keep waiting:

Condition 1: `last_to_enter[l] == i`

"Am I the last one to arrive at this level?" If yes, you're the "loser" — you must defer to others. If another process arrives after you and overwrites last_to_enter[l], you're freed.

Condition 2: `∃ k≠i : level[k] ≥ l`

"Is there anyone else at this level or above?" If all other processes have either not reached this level yet, or have finished (level[k] = -1), you're free to advance.

If EITHER condition becomes false → process i advances to the next level

Walkthrough: 3 processes

Step	Action	Result
1	`P0 sets level[0]=0, lte[0]=0`	P0 enters level 0
2	`P1 sets level[1]=0, lte[0]=1`	P1 enters level 0, overwrites lte[0] → P0 freed
3	`P0 passes gate 0, sets level[0]=1, lte[1]=0`	P0 advances to level 1
4	`P2 sets level[2]=0, lte[0]=2`	P2 enters level 0, overwrites lte[0] → P1 freed
5	`P1 passes gate 0, sets level[1]=1, lte[1]=1`	P1 advances to level 1, overwrites lte[1] → P0 freed
6	`P0 passes gate 1 → enters CS`	Only P0 in critical section

This is just one possible interleaving — CSP and FDR will explore all of them for you!

Modelling in CSP: key decisions

Shared variables → processes

CSP has no shared memory! Each variable (level[i], last_to_enter[l]) must be modelled as a separate process that accepts read/write events on channels.

WHILE loop → recursion

A spin-wait becomes a recursive process that reads variables, checks conditions, and either recurses (keep waiting) or proceeds (advance).

FOR loop → sequential composition

Each level iteration is a sequential chain: write level → write lte → spin-wait → next level. Can be parameterised with level number l.

System composition

PROC(0) ||| PROC(1) ||| PROC(2) interleaved, synchronised on shared register channels using alphabetised parallel.

Atomic registers in CSP

Each shared variable becomes a process that holds a value and offers read/write events.

```
-- A register holding value v
-- Offers: read (returns current value)
--         write (accepts new value)

channel read, write : RegisterID . Values

REG(id, v) =
    read.id.v -> REG(id, v)     -- return v, stay same
    []
    write.id?new_v -> REG(id, new_v)  -- accept new

-- External choice: the environment decides
-- whether to read or write
```

This is the fundamental building block — get this right first!

What to check in FDR

Mutual exclusion

```
-- Spec: at most one in CS
MUTEX_SPEC =
  enter_cs?p ->
    exit_cs.p ->
      MUTEX_SPEC

assert MUTEX_SPEC [T= SYSTEM_CS
```

Traces refinement: SYSTEM can only do traces that MUTEX_SPEC allows

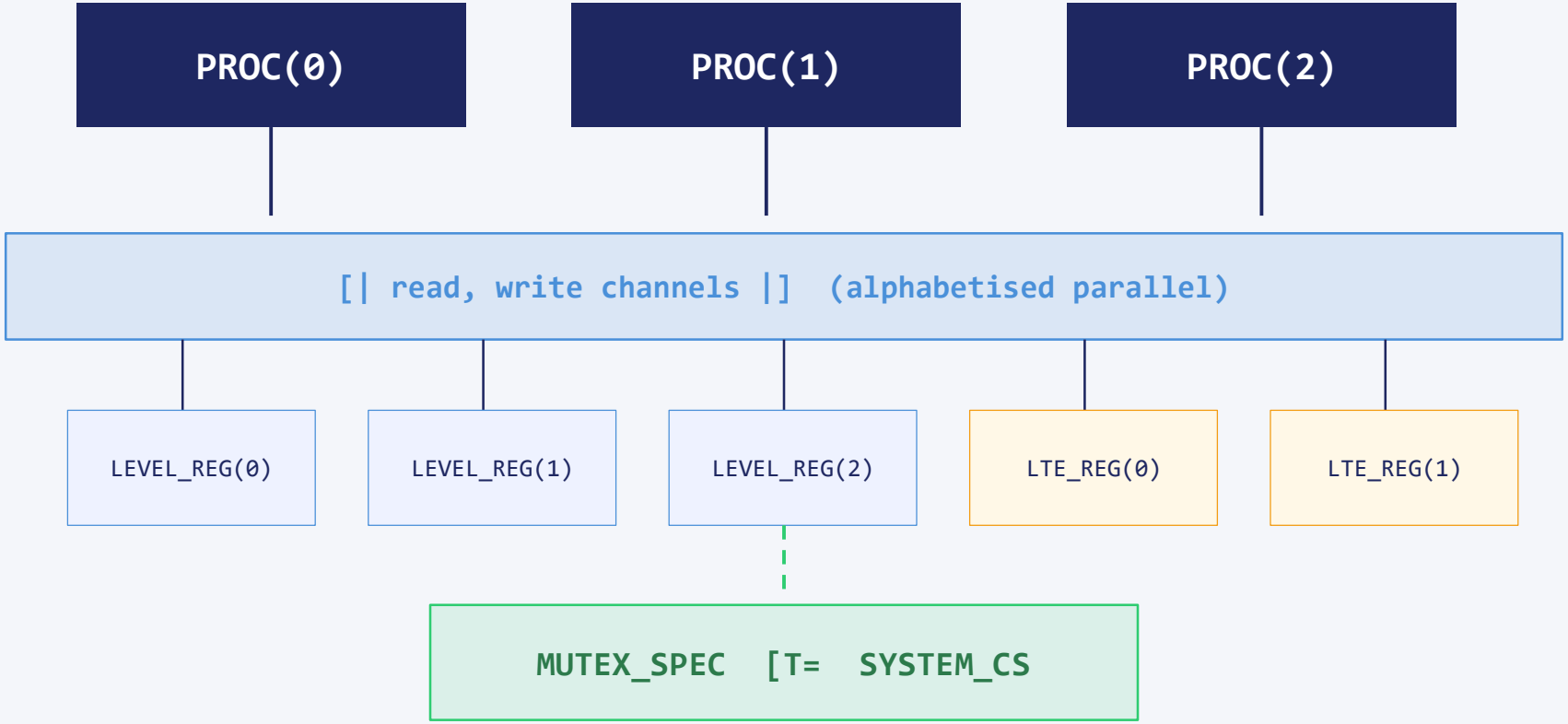
Progress (no blocking)

```
-- Check deadlock freedom
assert SYSTEM
  :[deadlock free]

-- Check divergence freedom
assert SYSTEM
  :[divergence free]
```

The algorithm must not block any process from eventually entering the CS (assuming CS doesn't block)

Architecture of your CSP-M model



Practical tips for your model

Start with 2 processes

Get Peterson's 2-process version working first. Then generalise to N=3.

Test registers in isolation

Load just REG(id, v) in FDR and use Probe to verify read/write behaviour before composing.

Use Probe for debugging

FDR's Probe tool lets you step through traces interactively — invaluable for finding where your model diverges from the algorithm.

Watch state space

With 3 processes, 3 level registers, and 2 lte registers, the state space grows fast. Keep variable domains small.

Name your channels well

Use descriptive channels: read_level.i.v, write_lte.l.i — you'll thank yourself when debugging traces.

Summary

- Peterson's algorithm generalises mutual exclusion to N processes
- It uses N−1 levels as a filtering queue (one process eliminated per level)
- In CSP: shared variables become register processes, loops become recursion
- Verify mutual exclusion via traces refinement in FDR
- Verify progress via deadlock-freedom and divergence-freedom checks

Good luck with Exercise 6, Task 2!